

Camera Calibration & Deployment Pipeline

Digit-reading ESP32-CAM network — calibration webpage, LoRa deployment, and Raspberry Pi ingest.

1. What this system does

Each camera reads a physical numeric display (a gauge, meter, or panel with a digital readout) and reports the digits it sees back to a central Raspberry Pi, wirelessly, without any camera-side WiFi needed once deployed. The pipeline has three firmware roles plus one Pi-side listener, and two very different phases per camera: a one-time calibration phase (WiFi + browser) where a human points the camera and checks its reading, followed by a permanent deployment phase (LoRa-only, unattended) where it just keeps reporting.

Role	File	Hardware	Job
Camera	<code>calibration_deployment_cam5.ino</code>	AI-Thinker ESP32-CAM	Captures frames, finds/locks onto the numeric display, black-and-white-thresholds it, optionally segments and classifies each digit on-device, and streams the result out over UART.
Sender (one per camera)	<code>calibration_deployment_lilyGoSender6_toPi.ino</code>	LilyGO T3 V1.6.1 (LoRa32)	Reads frames from its camera over UART, hosts a local calibration webpage over WiFi (its own mDNS name), relays browser commands straight to the camera, and — once you hit Deploy — drops WiFi and radios readings out over LoRa instead.
Gateway (one, shared)	<code>calibration_deployment_lilyGoReceiver6_toPi.ino</code>	LilyGO T3 V1.6.1 (LoRa32)	Listens for LoRa packets from every deployed sender, tags each by MAC address with an auto-assigned friendly name (Device-1, Device-2, ...), and prints one JSON line per reading over USB serial to the Pi.
Pi listener	<code>dynamicProv2.py</code>	Raspberry Pi	Reads the gateway's JSON lines over USB serial, keeps a live in-memory table of the latest reading per MAC, auto-provisions a brand-new ThingsBoard device the first time each MAC is seen (caching the resulting access token to disk), pushes each reading to ThingsBoard as telemetry, redraws a terminal dashboard, and exposes the latest readings over a small Flask API.

2. End-to-end data flow

- ESP32-CAM grabs a grayscale QVGA (320×240) frame roughly every 800 ms.
- It auto-detects the bright display panel, crops to it, black-and-white-thresholds the crop, and (once locked, if

enabled) segments individual digit blobs and classifies each with an on-device `DigitClassifier` — sorting left-to-right per row, with `.` inserted for decimal points and `\n` between rows.

- The result — a status byte, image size, packed 1-bit image data, and the recognized text — is sent over UART (`0xAA 0x55` sync header) to the LilyGO sender board wired to it.
- The sender board (`calibration_deployment_lilyGoSender6_toPi.ino`) holds the latest frame/text in RAM and serves it to a browser (calibration phase) via its own WebServer and mDNS name, e.g. `espcam-a1b2.local`.
- When you press Deploy on that camera's webpage, the sender tears down WiFi/mDNS permanently (until the next power-cycle) and switches to sending small LoRa packets instead — one per capture cycle — each carrying its own MAC address, the status byte, and the recognized text.
- The shared gateway board (`calibration_deployment_lilyGoReceiver6_toPi.ino`) listens for LoRa packets from any number of deployed senders, assigns each new MAC a friendly name the first time it's seen, and prints one JSON line per packet over USB serial, e.g.:

```
{"mac": "AA:BB:CC:DD:EE:FF", "name": "Device-1", "data":  
{"text": "42.7", "locked": true, "segmented": true, "recognized": true, "streaming": false}, "rssi": -63}
```

- The Raspberry Pi (running `dynamicProv2.py`) reads that serial port line by line and parses each line as JSON. For each MAC it has never seen before, it auto-provisions a new device in a locally running ThingsBoard instance and caches the returned access token to `device_tokens.json`; every reading is then pushed to that device's ThingsBoard telemetry endpoint over loopback (`http://localhost:8080` — no network needed for this last hop), while the terminal shows a live per-device dashboard and a small Flask API serves the latest readings at `/status`.

Key design point: calibration and deployment are two totally separate radio phases on the sender board. WiFi/webpage only exists before Deploy; LoRa-only reporting only exists after. Going back to calibration requires physically power-cycling that sender board.

3. Protocols (for reference / debugging)

3.1 Camera → Sender (UART, 115200 baud)

Frame packet:

```
0xAA 0x55 | status(1) | panelThresh(1) | w(2) | h(2) | packed 1-bit image data | textLen(1) | text(textLen)
```

Status byte bits: `0x01` locked · `0x02` segmented (this frame) · `0x04` recognized (this frame, non-empty text) · `0x08` streaming (raw full-frame preview mode).

Command byte (Sender → Camera, prefixed `0xC0`):

Value	Command	Effect
1	LOCK	Freezes onto the last auto-detected crop; resets segment/recognize to off
2	UNLOCK	Back to auto-searching for the panel; also used to leave STREAM mode
3 / 4	SEGMENT_ON / OFF	Toggle per-digit blob segmentation (locked mode only)
5 / 6	RECOGNIZE_ON / OFF	Toggle on-device digit classification (locked mode only)
8	STREAM	Raw full-frame BW preview, no detection — the camera's boot default
9	SET_PANEL_THRESH	One value byte follows; sets the brightness cutoff for panel detection
10 / 11	SHOW_MASK_ON / OFF	Preview the panel mask instead of the BW crop while searching/streaming

3.2 Sender ↔ Browser (HTTP, calibration phase only)

- `GET /` — the calibration webpage (buttons + live preview)
- `GET /frame.hex` — latest frame as `w,h,<hex bytes>`

- `GET /status` — JSON: `frameReady, locked, segmented, recognized, streaming, panelThresh, mac, text, loraReady`
- `GET /cmd?action=...` — one of `lock · unlock · segment_on · segment_off · recognize_on · recognize_off · stream · deploy · set_panel_thresh` (needs `&value=0-255`); relayed to the camera immediately, except `deploy` which is handled locally on the sender.

3.3 Sender → Gateway (LoRa, 868 MHz, deployment phase only)

```
0x03 | mac(6) | status(1) | textLen(1) | text(textLen)
```

Radio settings that must match on both boards: SF7, 125 kHz bandwidth, coding rate 4/5, sync word `0xF3`, 868 MHz (or 915 MHz — whatever your module's silkscreen says).

3.4 Gateway → Pi (USB serial, JSON lines)

```
{"mac":"AA:BB:CC:DD:EE:FF","name":"Device-1","data":
{"text":"...", "locked":true, "segmented":true, "recognized":true, "streaming":...}, "rssi":...}
```

This is the only thing the gateway ever prints on its main Serial line, so the Pi script reads it line-by-line and `json.loads()` s each line — nothing else should share that UART.

3.5 Pi ↔ ThingsBoard / Pi's own API

- **Provisioning** (first time a MAC is seen): `POST {THINGSBOARD_URL}/api/v1/provision` with `deviceName`, `provisionDeviceKey`, `provisionDeviceSecret`, `credentialsType=ACCESS_TOKEN` → returns an access token, cached in `device_tokens.json` so it's never re-provisioned.
- **Telemetry** (every reading): `POST {THINGSBOARD_URL}/api/v1/{token}/telemetry` with the reading's `data` object as the JSON body.
- **Pi's own status endpoint**: `GET http://<pi-ip>:5000/status` — returns the latest reading per MAC as JSON (bound to `0.0.0.0`, so it's reachable from other machines on the LAN).

4. Hardware & software you need before you start

This section is everything a new laptop needs — from a blank Arduino IDE install — to be able to flash all three firmware roles and run the Pi listener. Do this once per development machine.

4.1 Boards / parts shopping list

Part	Quantity	Used as	Notes
AI-Thinker ESP32-CAM	1 per camera	Camera role	Has no onboard USB — needs an external USB-to-TTL adapter to flash (see 4.2). Needs a board with real PSRAM (AI-Thinker module has 4MB PSRAM on the OV2640 camera module) — the firmware halts at boot if PSRAM allocation fails.
LilyGO T3 V1.6.1 (LoRa32, SX1276 or SX1278, 868/915 MHz)	1 per camera (sender) + 1 shared (gateway)	Sender / Gateway roles	Has onboard USB (flash directly, no external adapter needed), onboard LoRa radio, and an onboard SSD1306 OLED footprint. Confirm your module's frequency (868 vs 915 MHz) matches local regulations and matches on every board.
USB-to-TTL / FTDI-style serial adapter (e.g. CP2102, CH340, or FT232-based “USB-UART” breakout)	1 (reusable across all cams)	Flashing the ESP32-CAM only	The ESP32-CAM board itself has no USB port. Wire adapter GND→GND, 5V→5V, TXD→cam U0R, RXD→cam U0T, and pull GPIO0 to GND while powering on to enter flash mode.
Micro-USB or USB-C cables	1 per LilyGO board + 1 for the adapter	Flashing / power	LilyGO T3 V1.6.1 typically uses micro-USB.
Raspberry Pi (any model that runs Raspberry Pi OS / Debian, e.g. Pi 4/5)	1	Pi listener + ThingsBoard	Needs a free USB port for the gateway board.
Numeric display / gauge to read	1 per camera	The thing being monitored	Any backlit or well-lit digital readout the camera can be pointed at.

4.2 Install USB-to-serial drivers (on your laptop)

Your laptop needs a driver for whichever USB-to-serial chip is on your programmer, so the board shows up as a COM port (Windows) or `/dev/tty.*` / `/dev/ttyUSB*` device (macOS/Linux). Check the chip printed on the adapter (or in Device Manager once plugged in) and install the matching driver — installing the wrong one silently does nothing, so match it to the actual chip:

Chip	Common on	Driver download
CP2102 / CP2104 (Silicon Labs)	Many ESP32-CAM programmer boards, some LilyGO boards	Silicon Labs “CP210x USB to UART Bridge VCP Drivers” (silabs.com)
CH340 / CH340C / CH341	Cheap generic USB-TTL adapters, many ESP32/ESP8266 dev boards	WCH “CH340 driver” (search “CH340 driver” — WCH’s official site hosts Windows/Mac/Linux versions)
FT232R / FTDI	Some higher-quality USB-TTL adapters	FTDI “VCP Driver” (ftdichip.com)

Notes: - **Windows:** download and run the installer for the matching chip, then unplug/replug the board and confirm a COM port appears in Device Manager. - **macOS:** modern macOS (Big Sur+) has CH340 support built in for many revisions; if the port doesn’t appear, install the vendor’s signed macOS driver and allow it in System Settings → Privacy & Security. - **Linux:** CP210x and CH340 are usually already in the kernel (`cp210x` / `ch341` modules) — just make sure your user is in the `dialout` group (`sudo usermod -aG dialout $USER` , then log out/in) so you have permission to open `/dev/ttyUSB*` . - The LilyGO T3 boards’ onboard USB chip typically also needs one of the drivers above (commonly CP2104) — check its silkscreen/datasheet if it doesn’t enumerate.

4.3 Install Arduino IDE + the ESP32 board package

1. Install the Arduino IDE (2.x recommended) from arduino.cc.
2. Open File → Preferences, and add this URL to “Additional Boards Manager URLs”:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
3. Open Tools → Board → Boards Manager, search “esp32”, and install the **esp32 by Espressif Systems** package.

- Select the board for each `.ino` file before compiling/flashing:
 - `calibration_deployment_cam5.ino` → Tools → Board → ESP32 Arduino → **AI Thinker ESP32-CAM**. Also set Tools → Partition Scheme → **Huge APP (3MB No OTA/1MB SPIFFS)** or a scheme with enough app space, and confirm **PSRAM: Enabled**.
 - `calibration_deployment_lilyGoSender6_toPi.ino` and `calibration_deployment_lilyGoReceiver6_toPi.ino` → Tools → Board → ESP32 Arduino → **ESP32 Dev Module** (this is what was used for both the LilyGO sender and receiver boards).
- Pick the right Port (the COM port / `/dev/tty*` your driver from 4.2 exposed) for whichever board is currently plugged in.

4.4 Install required Arduino libraries

Install these via Sketch → Include Library → Manage Libraries (search by name):

Library	Needed by	Notes
LoRa by Sandeep Mistry	Sender, Gateway	SX1276/SX1278 LoRa driver.
Adafruit SSD1306	Sender	OLED display driver.
Adafruit GFX Library	Sender	Dependency of Adafruit SSD1306 (installs alongside it, or accept the prompt to install dependencies).
ESP32 core built-ins: <code>WiFi.h</code> , <code>WebServer.h</code> , <code>ESPmDNS.h</code> , <code>Wire.h</code> , <code>esp_camera.h</code> , <code>esp_heap_caps.h</code> , <code>esp_mac.h</code>	Sender / Camera	These ship with the esp32 board package from 4.3 — no separate install needed.

The camera board also needs its project-local headers (`board_config.h`, `digit_classifier_2.h`) — these come with the camera firmware’s source folder, so make sure the `.ino` and its headers stay together in the same sketch folder when you open it in Arduino IDE.

4.5 Install Python dependencies (Raspberry Pi, or a laptop for testing)

```
pip install flask pyserial requests
```

`dynamicProv2.py` targets Python 3. If you want to run/test the Pi listener from your laptop instead of an actual Pi, plug the gateway board into the laptop, update `SERIAL_PORT` to match (see 4.2 for finding it), and everything else works the same — just remember ThingsBoard also needs to be reachable wherever you point `THINGSBOARD_URL`.

4.6 Set up ThingsBoard (one-time)

ThingsBoard needs to be running and reachable at whatever `THINGSBOARD_URL` you configure in `dynamicProv2.py` (the default in this project is `http://localhost:8080`, i.e. running on the same machine as the Pi listener). The easiest way to get a local instance running on a Pi or laptop is via Docker — see ThingsBoard’s official “Install on Docker” guide for the current commands for your OS. Once it’s running, follow section 4.7 below to configure device provisioning.

4.7 One-time ThingsBoard device profile setup

- In the ThingsBoard UI: Profiles → Device profiles, open (or create) a device profile.
- Open its Provisioning tab and set the strategy to **“Allow to create new devices”**.
- Copy the generated **Provision device key** and **Provision device secret** into `PROVISION_DEVICE_KEY` / `PROVISION_DEVICE_SECRET` near the top of `dynamicProv2.py` — both are placeholders (`"REPLACE_ME"`) until you do this.

5. Step-by-step: setting up and running the pipeline

5.1 One-time hardware wiring (per camera unit)

- Wire the ESP32-CAM’s UART TX to the sender’s UART RX (GPIO34), and the sender’s UART TX (GPIO12) to the camera’s UART RX (GPIO3 / U0R).
 - Important:** GPIO3/U0R on the ESP32-CAM is shared with the USB-serial flashing line — disconnect this wire before reflashing the camera board.
- Wire the sender’s LoRa module: SCK 5, MISO 19, MOSI 27, SS 18, RST 23, DIO0 26 (LilyGO T3 V1.6.1 / LoRa32 V2.1.6

onboard pins — no extra wiring needed if using the integrated board).

- Optional: connect an SSD1306 OLED to the sender over I2C (SDA 21 / SCL 22) — shows the mDNS name, IP, and MAC at boot; the board still works fine without one.
- The gateway board only needs its onboard LoRa radio and a USB connection to the Raspberry Pi.

5.2 Flash the firmware

1. In Arduino IDE, set `WIFI_SSID` / `WIFI_PASSWORD` near the top of `calibration_deployment_lilyGoSender6_toPi.ino` to your network's credentials before flashing each sender board.
2. Flash `calibration_deployment_cam5.ino` to each ESP32-CAM (with the UART-RX wire to the sender disconnected during flashing, using the external USB-to-TTL adapter from 4.1/4.2, and holding/pulling GPIO0 low to enter flash mode).
3. Flash `calibration_deployment_lilyGoSender6_toPi.ino` to each LilyGO board that's paired with a camera — one sender per camera.
4. Flash `calibration_deployment_lilyGoReceiver6_toPi.ino` to a single, separate LilyGO board — this is the shared gateway; only one is needed no matter how many cameras you deploy.

5.3 Calibrate each camera

1. Power up the camera + sender pair. It boots into `MODE_STREAM` (raw preview) and connects to WiFi.
2. Read its address off the OLED (or Serial monitor at 115200 baud): `espcam-xxxx.local`, derived automatically from that board's MAC — no manual per-board setup needed.
3. Open that address in a browser. You'll see a live BW preview and buttons for Stream / Lock / Unlock / Segment / Recognize / a panel-threshold slider / Deploy.
4. Adjust the panel threshold slider until the detector reliably finds the numeric display, then hit Lock to freeze the crop.
5. Turn on Recognize and check the "Reading:" text against the real display. Toggle Segment on too if you want to see the individual digit boxes it's using.
6. Repeat for every camera — each one gets its own browser tab / address, so you can calibrate several in parallel.
7. The Deploy button only becomes active once the camera is both locked and recognizing (so you can't deploy a camera that isn't actually reading anything yet).

5.4 Deploy

1. Once a camera's reading looks correct, click Deploy on its webpage.
2. That sender board immediately drops WiFi/mDNS for good (until next power-cycle) and starts radioing LoRa packets — one per ~800 ms capture cycle — tagged with its own MAC address.
3. This is one-way by design: to recalibrate that camera later, physically power-cycle its sender board, which boots straight back into WiFi/calibration mode.

5.5 Run the Pi-side listener

1. Connect the gateway board to the Raspberry Pi over USB.
2. Find its serial port: on Linux, `ls /dev/serial/by-id/` (most reliable, stable name) or `ls /dev/ttyUSB* /dev/ttyACM*`. Set `SERIAL_PORT` in the script to match (defaults to `/dev/ttyUSB0`).
3. Install dependencies: `pip install flask pyserial requests`.
4. Run it: `python3 dynamicProv2.py`.
5. The terminal redraws a live table (device name, MAC, RSSI, last-seen time, and each data field) every time a new reading arrives. The first reading from a brand-new MAC triggers ThingsBoard provisioning automatically — no manual device creation needed per camera.
6. As more cameras are deployed, neither the gateway nor this script need reconfiguring — both name/provision newly-seen MACs automatically. Note the gateway's own friendly names (Device-1, Device-2...) reset on gateway reboot, but the Pi's ThingsBoard device mapping in `device_tokens.json` persists across Pi restarts regardless.

6. Troubleshooting quick reference

Symptom	Likely cause / fix
Camera board hangs at boot, never streams	PSRAM allocation or digit-classifier init failed (no PSRAM available, or wrong flash-size/PSRAM setting in Arduino IDE board config). Needs ~1 MB free PSRAM plus a small internal-RAM tensor arena.
Can't reach <code>espcam-xxxx.local</code>	mDNS may be blocked on your network — use the raw IP shown on the OLED/Serial log instead, or check your router's connected-clients list.
Webpage loads but no image / "Waiting for frame"	Check the camera↔sender UART wiring (TX/RX not swapped) and that the camera's RX wire isn't still connected to USB from a recent reflash.
Deploy button greyed out	Camera must be both Locked and Recognizing first (recognized text non-empty).
"This board's LoRa radio isn't responding" on the webpage	SPI wiring to the LoRa module or <code>LoRa.begin()</code> failed — Deploy is disabled until this is fixed; check the <code>LORA_*</code> pin defines match your board revision.
Gateway prints nothing on serial	No packets received yet, LoRa init failed on the gateway (it halts if <code>LoRa.begin()</code> fails), or a sync-word/frequency/SF/BW mismatch between gateway and sender.
Multiple cameras' readings show under the same "Device-N" name	Shouldn't happen (naming is per-MAC) unless the gateway rebooted and renumbered — names aren't persisted across gateway reboots.
Pi script prints "Serial error", retries every 3s	<code>SERIAL_PORT</code> doesn't match the gateway's actual port, the gateway isn't plugged in, or another process has the port open. Re-check with <code>ls /dev/serial/by-id/</code> .
"Provisioning rejected" / no token for a MAC	<code>PROVISION_DEVICE_KEY</code> / <code>SECRET</code> are still <code>"REPLACE_ME"</code> , the device profile's provisioning strategy isn't set to "Allow to create new devices", or ThingsBoard isn't running on <code>localhost:8080</code> .
ThingsBoard telemetry never updates for a device	Check the cached token in <code>device_tokens.json</code> is still valid in ThingsBoard (e.g. device wasn't deleted there); delete its entry from that file to force re-provisioning.
Board doesn't show up as a COM port / <code>/dev/tty*</code> on your laptop	The USB-to-serial driver for that board's chip (CP210x / CH340 / FTDI — see section 4.2) isn't installed, or you're using a charge-only USB cable instead of a data cable.
ESP32-CAM won't enter flash mode (upload fails / times out)	GPIO0 must be pulled to GND before/during power-up or reset to enter flashing mode, and the UART-RX wire to the sender board must be disconnected first (it shares the same pin as USB-serial RX).

Generated from: `calibration_deployment_cam5.ino`, `calibration_deployment_lilyGoSender6_toPi.ino`, `calibration_deployment_lilyGoReciever6_toPi.ino`, and `dynamicProv2.py`.